

Lekce 11: Volný projekt

Plánovací list

Dnes si programuješ, co chceš — použij vše, co jsi se naučil/a. Nejdřív si ale projekt naplánuj.

Úkol 1: Plán projektu

Název projektu:

Co bude micro:bit dělat? (1–3 věty)

Jaké koncepty z kurzu použiješ? (zaškrtni alespoň 3)

☐ proměnné (int, str, float)	☐ podmínky (if/elif/else)	☐ cyklus (while/for)	☐ random
☐ temperature()	☐ accelerometer	☐ seznamy	☐ music
☐ funkce (def)	☐ řetězcové operace	☐ radio	☐ jiné:

Jaké vstupy bude program mít? (tlačítka, třepání, teplota, radio...)

Jaké výstupy bude program mít? (display, scroll, zvuk, radio...)

Jaké funkce (def) nadefinuješ? (alespoň 1)

Úkol 2: Skica programu

Napiš **pseudokód nebo kostru programu** — hlavní části, ne nutně funkční kód.

```
from microbit import *
# import dalsich modulu...

# definice funkci...

while True:
    # hlavni smycka...
    pass
```

Úkol 3: Realizace a reflexe

Po dokončení projektu odpověz:

1. Co se ti povedlo? Co jsi musel/a upravit oproti plánu?

2. Jaký koncept byl nejtěžší? Kde jsi potřeboval/a pomoc?
3. Co bys přidal/a, kdybys měl/a více času?

Výsledný program ulož a připrav k odevzdání do Google Classroom spolu s krátkým popisem (2–4 věty).

★ Bonus: Výzvy pro rychlé

Jsi hotov/a dřív než ostatní? Rozšiř svůj projekt jednou z těchto výzev:

Výzva A — Rekord (highscore):

Přidej proměnnou, která si pamatuje nejlepší výsledek a ukáže ho po stisku B.

```
rekord = 0
# ...
if skore > rekord:
    rekord = skore
    display.show(Image.HAPPY)
```

Výzva B — Odpočítávání před startem:

Na začátku programu zobraz odpočet 3–2–1 a teprve pak spust hlavní smyčku.

```
for i in range(3, 0, -1):
    display.show(str(i))
    sleep(700)
display.show(Image.YES)
```

Výzva C — Konfigurace přes tlačítka:

Přidej „nastavovací“ obrazovku: tlačítkem A/B nastavíš parametr programu (rychlost, obtížnost, počet pokusů) a teprve pak program spustíš.

Vyber si jednu výzvu, implementuj ji a krátce popiš, co jsi přidal/a.

★ Bonus: Kulička na plošince

Dosud jsme zobrazovali obrázky nebo znaky. Micro:bit ale umí rozsvítit i **jednu konkrétní LED** příkazem `display.set_pixel(x, y, jas)`, kde `x` a `y` jsou souřadnice (0–4) a `jas` je 0–9.

Chceme nakloněním desky posouvat tečku po displeji — jako kulička na plošince. Akce-

lerometr vrací náklon v rozsahu -1024 až 1024 . Musíme ho **přepočítat** na souřadnici 0–4. Takový přepočet se nazývá **mapování**.

Náklon (vstup)	Souřadnice LED (výstup)
-1024	0 (levý / horní kraj)
0	2 (střed)
$+1024$	4 (pravý / dolní kraj)

```
from microbit import *

def mapuj(hodnota, vstup_min, vstup_max, vystup_min, vystup_max):
    # Přepočítá hodnotu z jednoho rozsahu do druhého
    pomer = (hodnota - vstup_min) / (vstup_max - vstup_min)
    return int(pomer * (vystup_max - vystup_min) + vystup_min)

def orez(hodnota, minimum, maximum):
    # Ořízne hodnotu do povolených mezí
    if hodnota < minimum:
        return minimum
    if hodnota > maximum:
        return maximum
    return hodnota

while True:
    x = accelerometer.get_x()
    y = accelerometer.get_y()

    px = mapuj(x, -1024, 1024, 0, 4)
    py = mapuj(y, -1024, 1024, 0, 4)
    px = orez(px, 0, 4)
    py = orez(py, 0, 4)

    display.clear()
    display.set_pixel(px, py, 9)
    sleep(50)
```

1. Zadej program a naklánej micro:bitem — pohybuje se světelná tečka?
2. Co dělá funkce `mapuj()`? Proč nestačí prostý součin?
3. Co by se stalo bez `orez()`? Kdy by to způsobilo chybu?
4. **Výzva — fyzikálnější kulička:** Tečka se teď pohybuje přesně podle náklonu. Skutečná kulička by ale nabírala rychlost postupně — náklon by měnil *rychlost*, ne rovnou polohu. Zkus tuto variantu:

```
kx = 2.0    # počáteční poloha: střed
ky = 2.0

while True:
    kx += accelerometer.get_x() / 2000    # tilt mení rychlost
    ky += accelerometer.get_y() / 2000

    kx = orez(kx, 0.0, 4.0)
    ky = orez(ky, 0.0, 4.0)
```

```
display.clear()
display.set_pixel(int(kx), int(ky), 9)
sleep(50)
```

Jak se chování liší od původního programu? Proč se tečka pohybuje i když držíš micro:bit rovně (po předchozím náklonu)?